Pràctica 4: Optimització en SAT

Lògica en la Informàtica

FIB

Antoni Lozano

Q1 2026–2027

Objectius

Aquesta pràctica té com a objectiu:

- Fer servir *SAT solvers* per optimitzar problemes combinatoris.
- Concretament, seguint l'exemple de **minColoring**, resoldre **factory** i **towers**. Per a **factory**, s'adjunten tres exemples d'entrada. Del que té *maybe* al nom, no en coneixem l'òptim.

Referències

Com a guia d'estudi teniu

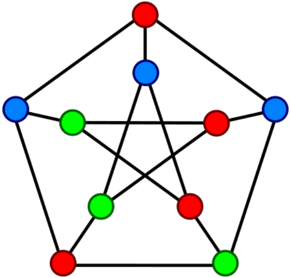
- l'exemple del **minColoring**
- aquestes transparències

Min Coloring

Coloració i nombre cromàtic

Una coloració (dels vèrtexs) d'un graf no dirigit G és una assignació d'etiquetes de colors als vèrtexs de G tal que cap aresta connecta dos vèrtexs amb el mateix color.

El nombre mínim de colors de qualsevol coloració de G se'n diu nombre cromàtic de G i es representa amb $\chi(G)$.



Graf de Petersen

Min Coloring

L'objectiu és trobar el nombre cromàtic d'un graf donat en el format:

```
%%%%%%% Begin example input %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

numNodes(15).
adjacency(1, [  2,3,4,5,6,    9,10,11,12,13,14,15]).
adjacency(2, [1,  3,4,5,6,7,  9,   11,12,      15]).
adjacency(3, [1,2,  4,5,6,7,8,9,10,   12,13,14   ]).
adjacency(4, [1,2,3,  5,6,7,  9,10,11,12,13,   15]).
adjacency(5, [1,2,3,4,    7,8,9,      12,13,   15]).
adjacency(6, [1,2,3,4,      8,  10,11,         15]).
adjacency(7, [  2,3,4,5,    8,9,10,11,      14   ]).
adjacency(8, [    3,  5,6,7,  9,10,      13,14,15]).
adjacency(9, [1,2,3,4,5,  7,8,     11,         15]).
adjacency(10,[1,  3,4,  6,7,8,     11,12,   14,15]).
adjacency(11,[1,2,  4,  6,7,  9,10,   12,13,14   ]).
adjacency(12,[1,2,3,4,5,        10,11,   13,14,15]).
adjacency(13,[1,  3,4,5,    8,     11,12,   14,15]).
adjacency(14,[1,  3,      7,8,  10,11,12,13,   15]).
adjacency(15,[1,2,  4,5,6,  8,9,10,   12,13,14   ]).

%%%%%%% End example input %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

Esquema general

El nostre esquema per resoldre problemes d'optimització amb un *SAT solver* genera les clàusules, crida al *SAT solver*, mostra la solució i calcula el seu cost.

Només cal especificar:

1. Les variables SAT (`satVariable`)
2. Les clàusules que descriuen el problema (`writeClauses`)
3. El format de la solució (`displaySol`)
4. El càlcul del cost (`costOfThisSolution`)

Min Coloring

```
%%%%%%% Some helpful definitions to make the code cleaner: ==================

node(I):-   numNodes(N), between(1,N,I).
edge(I,J):- adjacency(I,L), member(J,L).
color(C):-  numNodes(N), between(1,N,C).

%%%%%%% End helpful definitions ============================================


%%%%%%%  1. Declare SAT variables to be used: ================================

% x(I,C)    meaning  "node I has color C"
satVariable( x(I,C) ):- node(I), color(C).
```

Min Coloring

```
%%%%%%  2. Clause generation for the SAT solver: ============================================

% This predicate writeClauses(MaxCost) generates the clauses that guarantee that
% a solution with cost at most MaxCost is found

writeClauses(infinite):- !, numNodes(N), writeClauses(N),!.
writeClauses(MaxColors):-
    eachNodeExactlyOnecolor(MaxColors),
    noAdjacentNodesWithSameColor(MaxColors),
    true,!.
writeClauses(_):- told, nl, write('writeClauses failed!'), nl,nl, halt.

eachNodeExactlyOnecolor(MaxColors):-
    node(I), findall(x(I,C), between(1,MaxColors,C), Lits ), exactly(1,Lits), fail.
eachNodeExactlyOnecolor(_).

noAdjacentNodesWithSameColor(MaxColors):-
    edge(I,J), between(1,MaxColors,C), writeOneClause([ -x(I,C), -x(J,C) ]), fail.
noAdjacentNodesWithSameColor(_).
```

Min Coloring

```
%%%%%%%  3. DisplaySol: this predicate displays a given solution M: ===========================

% displaySol(M):- nl, write(M), nl, nl, fail.
displaySol(M):- node(I), member(x(I,C),M), write(I-C), write(' '), fail.
displaySol(_):- nl,!.


%%%%%%%  4. This predicate computes the cost of a given solution M: ===========================

% Here the sort predicate is used to remove repeated elements of the list:
costOfThisSolution(M,Cost):- findall(C,member(x(_,C),M),L), sort(L,L1), length(L1,Cost), !.
```

Min Coloring

```
?- [minColoring].
true.

?- main(minColoringExampleA).
Looking for initial solution with arbitrary cost...
Generated 3840 clauses over 225 variables.
Launching kissat...

Solution found with cost 7

1-11 2-13 3-15 4-9 5-10 6-14 7-11 8-12 9-14 10-13 11-15 12-12 13-13 14-14 15-15


Now looking for solution with cost 6...
Generated 1140 clauses over 90 variables.
Launching kissat...

Solution found with cost 6

1-2 2-4 3-6 4-1 5-5 6-3 7-2 8-1 9-3 10-4 11-6 12-3 13-4 14-5 15-6


Now looking for solution with cost 5...
Generated 915 clauses over 75 variables.
Launching kissat...

Unsatisfiable. So the optimal solution was this one with cost 6:
1-2 2-4 3-6 4-1 5-5 6-3 7-2 8-1 9-3 10-4 11-6 12-3 13-4 14-5 15-6
```

Factory

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% We have a factory of concrete products (beams, walls, roofs) that
%% works permanently (168h/week).  Every week we plan our production
%% tasks for the following week.  For example, one task may be to produce
%% a concrete beam of a certain type, which takes 10 hours and requires
%% (always one single unit of) the following resources: platform, crane,
%% truck, mechanic, driver.  But there are only a limited amount of units
%% of each resource available. For example, we may have only 3 trucks.  We
%% have 168 hours (numbered from 1 to 168) for all tasks, but we want to
%% finish all tasks as soon as possible.
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

Factory

File **easy152.pl**:

```
maxHour(168).

%% task( taskID, Duration, ListOFResourcesUsed ).
task(1,19,[1,2]).
task(2,52,[1,2]).
task(3,16,[1,3]).
task(4,52,[1,3]).
task(5,16,[2,3]).
task(6,20,[2,3]).
task(7,45,[2,3]).

%% resourceUnits( resourceID, NumUnitsAvailable ).
resourceUnits(1,2).
resourceUnits(2,1).
resourceUnits(3,2).
```

Factory

```
%%%%%%% Some helpful definitions to make the code cleaner: ====================================

task(T) :-              task(T,_,_).
duration(T,D) :-        task(T,D,_).
usesResource(T,R) :-    task(T,_,L), member(R,L).
hour(H) :-              maxHour(M), between(1,M,H).

%%%%%%% End helpful definitions ===============================================================


%%%%%%%  1. Declare SAT variables to be used: =================================================

satVariable( start(T,H) ) :- task(T), hour(H).   % "task T starts at hour H"     (MANDATORY)
% more variables will be needed....
```

Factory

Caldrà generar clàusules per limitar els recursos. Per fer-ho, seria útil saber quines tasques estan actives cada hora (amb noves variables SAT).

```
%%%%%%  2. Clause generation for the SAT solver: ===============================

% This predicate writeClauses(MaxCost) generates the clauses that guarantee that
% a solution with cost at most MaxCost is found

writeClauses(infinite) :- !, maxHour(M), writeClauses(M), !.
writeClauses(MaxHours) :-
    eachTaskStartsOnce(MaxHours),
    ...
    true, !.
writeClauses(_) :- told, nl, write('writeClauses failed!'), nl,nl, halt.
```

Factory

```
%%%%%%%  3. DisplaySol: this predicate displays a given solution M: ===========================

% displaySol(M):- nl, write(M), nl, nl, fail.
% No need to modify displaySol:
displaySol(_):- nl,nl,   write('    '),
    write('        10        20        30        40        50        60        70        80'),
    write('        90       100       110       120       130       140       150       160'),nl, write('    '),
    write('12345678901234567890123456789012345678901234567890123456789012345678901234567890'),
    write('123456789012345678901234567890123456789012345678901234567890123456789012345678'),nl,fail.
displaySol(M):- task(T), writeNum2(T), member(start(T,H),M), duration(T,D),
                B is H-1, writeX(' ',B), writeX('x',D), nl, fail.
displaySol(M):- findall(T-H,member(start(T,H),M),L), sort(L,L1), write(startTimes(L1)), write('.'),  nl,nl,!.
displaySol(_).

writeX(_,0):-!.
writeX(X,N):- write(X), N1 is N-1, writeX(X,N1),!.

writeNum2(T):-T<10, write(' '), write(T), write(': '), !.
writeNum2(T):-                  write(T), write(': '), !.
```

Factory

El cost és l’hora màxima de finalització d’alguna tasca.

```
%%%%%%%  4. This predicate computes the cost of a given solution M: ======

costOfThisSolution(M,Cost) :-
    % 'Cost' is the maximum task completion hour of this solution/model M
    ...


%%%%%%% ==================================================================
```

Towers

```
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%% We are given a map with NxM positions. On the map are a number of square villages.
%% On each position inside a village we are allowed to place a tower, at most one tower per
%% village.  These towers can be used to watch horizontally and vertically (like the towers
%% or castles in the game of chess).
%% The aim is to determine where to position the towers such that each village (has at least
%% one position that) is watched by at least one tower (for example, if the village has a
%% tower itself), and MINIMIZE the total number of towers.
%% There is a list of "significant" villages that MUST have a tower.
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
```

rooks/torres

Condicions no explícites:

1. Les torres només es poden posar als pobles, no fora.
2. El nombre màxim de torres és `MaxNumTowers`
   (la variable apareix al predicat `writeClauses(MaxNumTowers)`)
3. Les que serveixin per relacionar diferents *SAT variables* entre si.

Towers

Inici de l'entrada `towersInput1`:

```
%%%%%%% begin input example towersInput1 %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
%%
%% upperLimitTowers(10).         % maxim number of towers allowed
%% significantVillages([e,q,r]). % list of villages where a tower is mandatory
%% map_size(34,40).              % numrows, numcols; the left upper corner of the map has coordinates (1,1)
%%
%% village(a, 2,20,2).           % village(ident,row,col,size):  row,col of left upper corner, and size of the square
%% village(b, 3,13,2).
%% village(c, 7,21,2).
%% village(d, 8,13,2).
%% village(e,10,30,4).
%% village(f,11, 7,3).
%% village(g,12, 3,2).
%% village(h,12,13,2).
%% village(i,12,20,2).
%% village(j,12,25,2).
%% village(k,12,37,3).
%% village(l,16, 2,2).
%% village(m,16, 6,3).
%% village(n,16,12,3).
%% village(o,16,20,3).
%% village(p,16,26,2).
```

Towers

Solució subòptima (cost 10):

```
     1.......10........20........30........40

 1   ........................................
 2   ...................aa...................
 3   ............bb.....aa...................
 4   ............bb..........................
 5   ........................................
 6   ........................................
 7   ....................cc..................
 8   ............dd......cc..................
 9   ............dd..........................
10   .............................eeee.......
11   ......fff....................eeee.......
12   ..gg..fff...hh.....ii...jj...eeee...kkk.
13   ..gg..fff...hh.....ii...jj...eeeT...kkk.
14   ....................................kkk.
15   ........................................
16   .ll..mmm...nnn.....ooo...pp....qqq......
17   .ll..mmm...nnn.....ooo...pT....qqq...rr.
18   .....mmm...nnn.....ooo.........qqT...rT.
19   ........................................
20   ........................................
21   ........................................
22   ....................Ts..................
23   ............tt......ss..................
24   ............tT..........................
25   ........................................
26   ........................................
27   ...................uuu..................
28   ...........vv......uuu..................
29   ...........vT......uuT..................
30   ........................................
31   ...........www..........................
32   ...........www..........................
33   ...........wwT......xx..................
34   ....................xT..................
```

Towers

```
%%%%%%% Some helpful definitions to make the code cleaner: ====================================

row(I):- map_size(N,_), between(1,N,I).                               % I is a row in the map
col(J):- map_size(_,M), between(1,M,J).                               % J is a col in the map
position(I,J):- row(I), col(J).                                       % I-J is a position in the map
village(V):- village(V,_,_,_).                                        % V is a village identifier
rowVillage(V,I):- village(V,I1,_,S), I2 is I1+S-1, between(I1,I2,I).  % I is a row of the village V
colVillage(V,J):- village(V,_,J1,S), J2 is J1+S-1, between(J1,J2,J).  % J is a col of the village V
posVillage(V,I,J):- rowVillage(V,I), colVillage(V,J).                 % I-J is one position in village V
significantVillage(V):- significantVillages(L), member(V,L).          % V is a significant village
posWatchesVillage(V,I,J):- position(I,J), rowVillage(V,I).            % position (I,J) watches village V
posWatchesVillage(V,I,J):- position(I,J), colVillage(V,J).            %               "

%%%%%%% End helpful definitions ================================================================
```

Towers

```
%%%%%%%  1. Declare SAT variables to be used: ===================================================

satVariable( towerPos(I,J) ):- row(I), col(J).   % means "there is a tower at position I-J"
% YOU MAY WANT TO INTRODUCE SOME OTHER VARIABLE FOR MAKING THE CARDINALITY CONSTRAINTS SMALLER


%%%%%%%  2. Clause generation for the SAT solver: ================================================

% This predicate writeClauses(MaxCost) generates the clauses that guarantee that
% a solution with cost at most MaxCost is found

writeClauses(infinite):- !, upperLimitTowers(N), writeClauses(N),!.
writeClauses(MaxNumTowers):-
    ...
    true,!.                    % this way you can comment out ANY previous line of writeClauses
writeClauses(_):- told, nl, write('writeClauses failed!'), nl,nl, halt.
```

Towers

```
%%%%%%%  3. DisplaySol: this predicate displays a given solution M: ===========================

displaySol(M):- write('    1........10........20........30........40'), nl,
		row(I), nl, write2(I), write('  '),  col(J), writePos(M,I,J), fail.
displaySol(_):- nl,nl.

writePos(M,I,J):- member(towerPos(I,J),M), write('T'), !.
writePos(_,I,J):- posVillage(V,I,J), write(V), !.
writePos(_,_,_):- write('.'), !.
write2(N):- N < 10, !, write(' '), write(N),!.
write2(N):- write(N),!.


%%%%%%%  4. This predicate computes the cost of a given solution M: ===========================

costOfThisSolution(M,Cost):- ...
```